

char类型

基础信息

- 字符型。
- 单位：一个字节（8 bit位）。
- 格式匹配符：
 - 数值型：
 - 有符号：%hhd —— char 显示数值专用格式匹配符。
 - 无符号：%hhu —— unsigned char 显示数值专用格式匹配符。
 - 字符型：
 - %c
- 取值范围：
 - 有符号：-128 ~ 127
 - 无符号：0 ~ 255
- 程序获取

```
1  #include <stdio.h>
2  #include <limits.h>
3
4  int main(void)
5  {
6      // 获取无符号数取值范围
7      printf("char 无符号 min = 0, max = %hhu\n", UCHAR_MAX);
8      // 获取有符号数取值范围
9      printf("char 有符号 min = %hhd, max = %hhd\n", CHAR_MIN, CHAR_MAX);
10     // 获取 char 占用的字节数
11     printf("char 大小 = %u\n", sizeof(char));
12     // 获取 unsigned char 占用的字节数
13     printf("unsigned char 大小 = %u\n", sizeof(unsigned char));
14
15     return 0;
16 }
17
```

```
#include <limits.h>

int main(void)
{
    // 获取无符号数取值范围
    printf("char 无符号 min = 0, max = %hu\n", UCHAR_MAX);
    // 获取有符号数取值范围
    printf("char 有符号 min = %hd, max = %hd\n", CHAR_MIN, CHAR_MAX);
    // 获取 char 占用的字节数
    printf("char 大小 = %u\n", sizeof(char));
    // 获取 unsigned char 占用的字节数
    printf("unsigned char 大小 = %u\n", sizeof(unsigned char));

    return 0;
}
```

Microsoft Visual Studio 调试控制台

char 无符号 min = 0, max = 255
char 有符号 min = -128, max = 127
char 大小 = 1
unsigned char 大小 = 1

ASCII码

- char 类型数据，数值 对应一个 ASCII 码。
- ASCII表。

ASCII表																										
(American Standard Code for Information Interchange 美国标准信息交换代码)																										
高四位 低四位		ASCII控制字符												ASCII打印字符												
		0000						0001						0010		0011		0100		0101		0110		0111		
		0						1						2		3		4		5		6		7		
		十进制	字符	Ctrl	代码	转义	字符	十进制	字符	Ctrl	代码	转义	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	十进制	字符	Ctrl
		0			0	@ NUL \0	空字符	16	▶	^P	DLE		数据链路转义	32		48	0	64	@	80	P	96	`	112	p	
		1	☺		1	^A SOH	标题开始	17	◀	^Q	DC1		设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q	
		2	☹		2	^B STX	正文开始	18	↕	^R	DC2		设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r	
		3	♥		3	^C ETX	正文结束	19	!!	^S	DC3		设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s	
		4	♦		4	^D EOT	传输结束	20	¶	^T	DC4		设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t	
		5	♣		5	^E ENQ	查询	21	§	^U	NAK		否定应答	37	%	53	5	69	E	85	U	101	e	117	u	
		6	♠		6	^F ACK	肯定应答	22	—	^V	SYN		同步空闲	38	&	54	6	70	F	86	V	102	f	118	v	
		7	•		7	^G BEL \a	响铃	23	↕	^W	ETB		传输块结束	39	'	55	7	71	G	87	W	103	g	119	w	
		8	▢		8	^H BS \b	退格	24	↑	^X	CAN		取消	40	(	56	8	72	H	88	X	104	h	120	x	
		9	○		9	^I HT \t	横向制表	25	↓	^Y	EM		介质结束	41	)	57	9	73	I	89	Y	105	i	121	y	
		A	◻		10	^J LF \n	换行	26	→	^Z	SUB		替代	42	*	58	:	74	J	90	Z	106	j	122	z	
		B	♂		11	^K VT \v	纵向制表	27	←	^[	ESC	le	溢出	43	+	59	;	75	K	91	[	107	k	123	{	
		C	♀		12	^L FF \f	换页	28	└	^_	FS		文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
		D	♪		13	^M CR \r	回车	29	↔	^_	GS		组分分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
		E	🎵		14	^N SO	移出	30	▲	^^	RS		记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~	
		F	🎵		15	^O SI	移入	31	▼	^.	US		单元分隔符	47	/	63	?	79	O	95	_	111	o	127	␣	
注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。																							Backspace 代码：DEL			
																							2013/08/08			

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char ch = 'A'; // 定义变量 ch，指定初值为 'A';
6
7     printf("ch = %c\n", ch); // c: character %c 用来显示字符的 格式匹配符。
8
9     ch = 'm'; // 给变量ch 赋值成 'm'，覆盖 原来的 'A';
```

```

10
11     printf("ch = %c\n", ch);
12
13     ch = 97;    // 使用 范围内的数值 97 ，给 ch 赋值。
14
15     printf("ch = %c\n", ch); // 将数值97，按照字符格式打印输出。
16
17     ch = 98;    // 使用 范围内的数值 98 ，给 ch 赋值。
18
19     printf("ch = %c\n", ch); // 将数值98，按照字符格式打印输出。
20
21     return 0;
22 }

```

练习：

- 将 大写字母，转换成 小写字母。

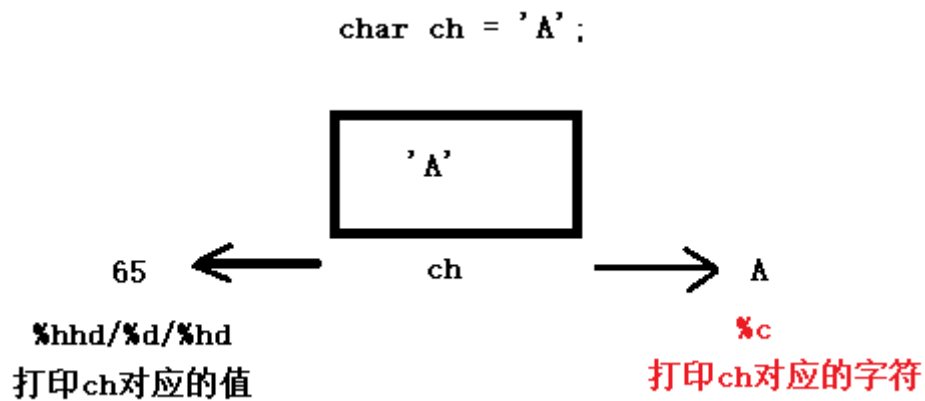
```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      char ch = 'R'; // char 变量定义
6
7      printf("R 转换的小写为: %c\n", ch+32); //ch+32 表达式，对应格式匹配符 %c
8
9      char ch2 = 'h'; // char 变量定义
10
11     printf("h 转换的大写为: %c\n", ch2-32); //ch2-32 表达式，对应格式匹配符 %c
12
13     char ch3 = '5';
14     // 借助字符5， 利用 ASCII特性，打印出 字符9
15     printf(" 打印字符9 = %c\n", ch3+4);
16
17     return 0;
18 }

```

ASCII表说明

- 0 ~ 32 ASCII码 对应的字符都不可见。
- 常用的 ASCII码：
 - 'a': ASCII码值 97
 - 'A': ASCII码值 65
 - '0': ASCII码值 48
 - '\n' ASCII码值 10
 - '\t' 制表符。tab键对应的字符。（ASCII码值 9）
 - '\0' ASCII码值 0



练习:

- 在一个printf函数中，打印输出 `hello` 换行 `world` 换行。

```
1 // 方法1
2 int main(void)
3 {
4     printf("hello\nworld\n");
5
6     return 0;
7 }
8 // 方法2
9 int main(void)
10 {
11     char ch = '\n'; //定义ch变量，初值为'\n'
12
13     printf("hello%cworld%c", ch, ch); // 等价于 printf("hello\nworld\n");
14
15     return 0;
16 }
```

- 一个printf中打印 `hello` (一个tab缩进) `world` 换行

```
1 int main(void)
2 {
3     char ch1 = '\t'; // 实现 一个tab缩进
4     char ch2 = '\n'; // 实现 一个换行。
5
6     printf("hello%cworld%c", ch1, ch2);
7
8     return 0;
9 }
```

转义字符

- `'/`: 自右向左划，叫做“斜杠”。
- `'\`: 自左向右划，叫做“反斜杠”。——是转义字符。
- 转义字符的作用：
 1. 将普通字符，转换为特殊意。

1. 如: '\n': 这是一个字符。代表换行。
 2. 如: '\t': 这是一个字符。代表 一个制表符。
2. 将特殊字符, 还原成本身意。
1. 如: '\\n': 这样就将 一个字符 '\n', 还原成 两个字符: '\' 和 'n'

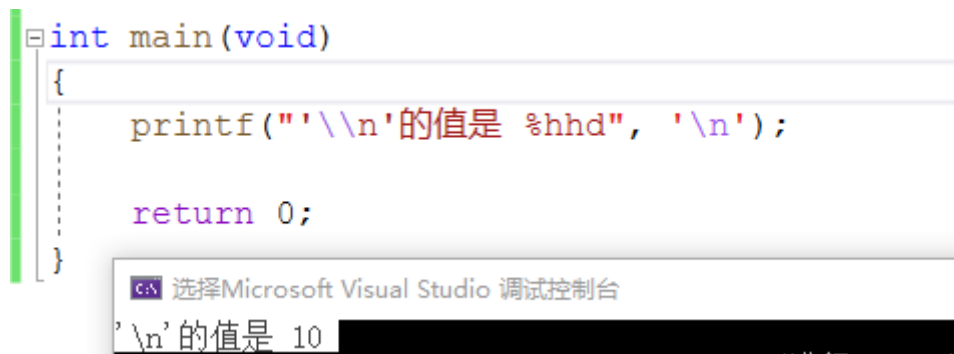
练习

- 将 特殊字符转换成本身意。

题目: 屏幕上严格输出 如下内容: 【'\n' 的值是 10】 要求显示时要有 ''

-

```
1 int main(void)
2 {
3     printf("'\\n'的值是 %hd", '\n');
4
5     return 0;
6 }
```



实型（浮点型）【了解】

- 显示小数

基础信息

- float: 单精度浮点型。%f 大小: 4字节。(可以使用 sizeof() 求取)
 - 4.35 默认会被编译器理解为 double 类型。
 - float v = 4.567f; 编译器理解 float。
 - %f 格式匹配符, 默认保留6位小数。
- double: 双精度浮点型。%lf 大小: 8字节。
 - double d = 5.68;

取值范围

- 使用头文件 #include <float.h> 获取浮点型取值范围。

- 求取的结果:

精度问题

- ```
int main(void)
{
 float f = 439243243.9f;

 printf("f = %f\n", f);

 return 0;
}
```

- ## bool 类型

- C语言原来没有 bool 类型。C99标准中，新增了 bool 类型。C++ 自带 bool 类型。

- 用处：
  - 表示：
    - 好、坏
    - 真、假
    - 对、错
    - 是、否
  - 取值：
    - true —— 真 —— 1
    - false —— 假 —— 0
- C语言使用bool 的条件：
  - 编译器要支持c99标准。
  - 导入 #include <stdbool.h>
- bool 类型的大小。
  - 1字节。（ sizeof() 求取 ）
- bool 没有专用的格式匹配符。打印时，使用 %d 来打印。
  - true —— 真 —— 1
  - false —— 假 —— 0

```
1 int main(void)
2 {
3 bool aa = true; // 定义bool 类型变量 aa , 初值为 true == 1
4
5 printf("aa = %d\n", aa);
6
7 aa = false; // 给 bool 类型变量 aa 赋值为 false == 0
8
9 printf("aa = %d\n", aa);
10
11 return 0;
12 }
```

## 进制和转换

---

- 计算机 只 使用 2 机制。
- 8进制、10进制、16进制。统统都是给人类用的!!!

## 存储知识

---

- 1 bit位 就是一个二进制位。存 0 或 1
- 一个字节 (Byte) 1B = 8bit位。
- “内存单元” 是计算机内存存储的最小单位， 一个内存单元 == 1字节。
- 1KB = 1024B
- 1MB = 1024KB
- 1GB = 1024MB
- 1TB = 1024GB

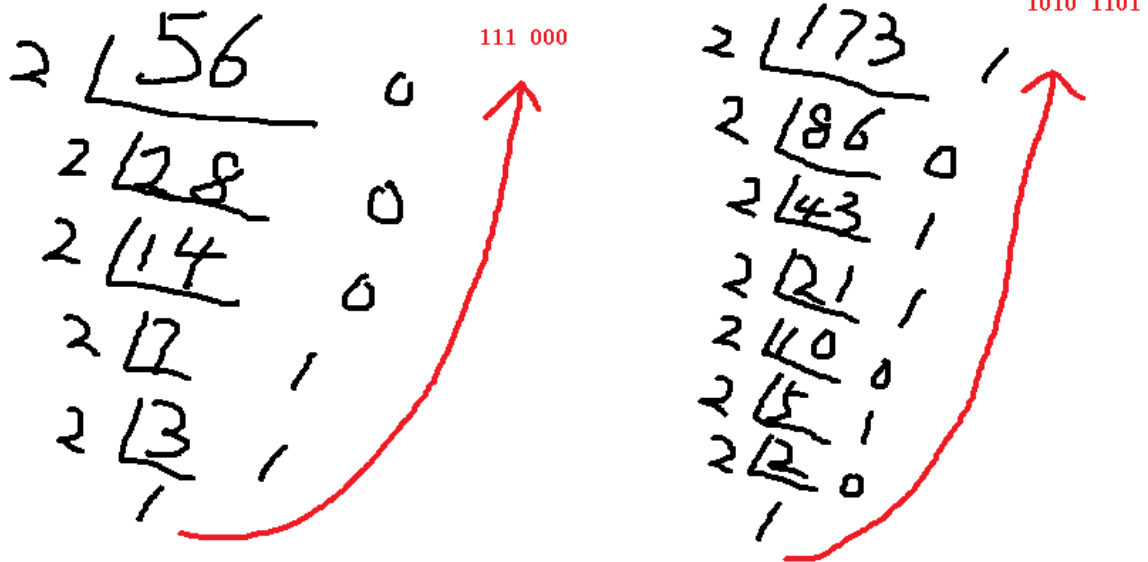
## 2进制

---

- 取值 1 或 0. 逢 2 进 1。借 1 当 2。
- 没有格式匹配符

## 10 转 2

- 除 2 反向取余法



## 2 转 10

- 掌握  $2^0 \sim 2^{10}$  各自的取值：
  - $2^0$  : 1
  - $2^1$  : 2
  - $2^2$  : 4
  - $2^3$  : 8
  - $2^4$  : 16
  - $2^5$  : 32
  - $2^6$  : 64
  - $2^7$  : 128
  - $2^8$  : 256
  - $2^9$  : 512
  - $2^{10}$  : 1024
- 计算方法，对应二进制位，有 1 累加 2 的指数次幂值，有 0 略过。



高 1875 低

高 1 1 0 1 0 1 0 1 1 低

$2^8 \quad 2^7 \quad 2^6 \quad 2^5 \quad 2^4 \quad 2^3 \quad 2^2 \quad 2^1 \quad 2^0$

$2^8 + 2^7 + 2^5 + 2^3 + 2^1 + 2^0$

256 128 32 8 2 1 = 427

$2^0$

$2^1$

$2^2$

$2^3$

$2^4$

$2^5$

$2^6$

$2^7$

$2^8$

- 11101 —— 29
- 10110 ——  $2+4+16 = 22$

## 8进制

- 逢 8 进 1。借 1 当 8。
- 取值：0/1/2/3/4/5/6/7 最大值 7
- 格式匹配符：%o 和 %#o
- 表示语法：0 开头。
- 8 进制 ——> 10 进制：
  - $056 == 8^0 * 6 + 8^1 * 5 = 6 + 40 = 46$
  - $0123 = 8^0 * 3 + 8^1 * 2 + 8^2 * 1 = 3 + 16 + 64 = 83$
- 10 进制 ——> 8 进制：
  - 除 8 反向取余法
    - 135 —— 0207

## 8 转 2

- 转换算法：将 8 进制数，自左向右，每一位，用 421 码 展开。
- 练习：
  - 056 —— 101110
  - 04735 —— 1001 1101 1101
  - 053261 —— 101 0110 1011 0001

## 2 转 8

- 转换方法：自右向左，每 3 个二进制位一组，不足 3 位补 0。
  - 1101010101111010
    - 分组：1 101 010 101 111 010

- 补齐: 001 101 010 101 111 010
- 八进制数为: 0152572
- 10110111011
  - 分组: 10 110 111 011
  - 补齐: 010 110 111 011
  - 八进制数为: 02673

## 16进制

---

- 逢 16 进 1。借 1 当 16。
- 取值: 0~9 10-A/a 11-B/b 12-C/c 13-D/d 14-E/e 15-F/f 最大值: F/f
- 格式匹配符: %x 或 %#x
- 表示语法: 0x 开头。
- 16 进制 ——> 10 进制:
  - 将每一个 16 进制位, 展开 相加得 10 进制。
- 10 进制 ——> 16 进制:
  - 除 16 反向取余法
    - 37: 0x25
    - 43: 0x2B

## 16 转 2

- 转换算法: 将 16 进制数, 自左向右, 每一位, 用 8421 码 展开
  - 0xFAC12 : 1111 1010 1100 0001 0010

## 2 转 16

- 转换方法: 自右向左, 每 4 个二进制位一组, 不足 4 位补 0。
  - 101001011101011001
    - 分组: 10 1001 0111 0101 1001
    - 补齐: 0010 1001 0111 0101 1001
    - 16 进制数为: 0x29759
  - 1110110100111101
    - 分组: 1110 1101 0011 1101
    - 补齐: 1110 1101 0011 1101
    - 16 进制数为: 0xED3D

## 小结

---

- 使用频率最高的是 10 进制。次高的: 16 进制。
  - int m = 0x15F4;
  - int n = 0345;
  - int k = 0101101;
  - 不能直接将变量值赋值为 2 进制。
  - 上述的赋值, 会被编译器理解为 8 进制数。

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5 int a = 56; // 10进制数作a变量初始化。 -- 定义。
6
7 printf("10进制: a = %d\n", a);
8 printf("8进制: a = %o\n", a);
9 printf("8进制: a = %#o\n", a); //在%和x中间添加#, 可以在输出时, 显示8进制前缀。
10 printf("16进制: a = %x\n", a);
11 printf("16进制: a = %#x\n", a); //在%和x中间添加#, 可以在输出时, 显示16进制前
 缀。
12
13 return 0;
14 }

```

## 常用的格式匹配符

- %d
- %c
- %x
- %u
- %s (后续课程讲, 打印字符串)
- %o
- %#o
- %#x
- %hhd
- %hd
- %ld
- %lld
- %hhu
- %hu
- %lu
- %llu
- %f
- %lf

## 编码和存储

### 无符号存储

```

1 unsigned int a = 12; // 占用 4 字节 32 个 bit位存储。
2 空间: 00000000 00000000 00000000 00000000 ----- 4 字节。
3 存储: 00000000 00000000 00000000 00001100
4
5 unsigned short b = 15; // 占用 2 字节 16 个 bit位存储。
6 空间: 00000000 00000000 ----- 2 字节。
7 存储: 00000000 00001111

```

## 有符号存储

- 需要拿出一个二进制位，专门存储符号。标识正、负。
  - 选用 最高位 为符号位。
  - 正：0
  - 负：1

## 有符号正数

```
1 // 采用“源码”存储
2 int a = 5;
3 空间: 0 0000000 00000000 00000000 00000000
4 存储: 0 0000000 00000000 00000000 00000101 --- 表示存储正数 5。
```

## 有符号负数

- 有符号的负数，采用“补码”存储
  - 源码：数值的二进制位直接存储。
  - 反码：符号位不变，将其余数值为取反。
  - 补码：反码+1。

```
1 int b = -33;
2 空间: 0 0000000 00000000 00000000 00000000
3 源码: 1 0000000 00000000 00000000 00100001
4 反码: 1 1111111 11111111 11111111 11011110
5 补码: 1 1111111 11111111 11111111 11011111 --- 负33 在计算机中实际的存储形式。
```

- 小结：
  - int —— 4字节 —— 32bit位
    - 有符号：31个数值位。取值范围  $-2^{31} \sim 2^{31}-1$  ——> -2147483648 ~ +2147483647
    - 无符号：32个数值位。取值范围  $0 \sim 2^{32}-1$  ——> 0 ~ 4294967295
  - short —— 2字节 —— 16bit位
    - 有符号：15个数值位。取值范围  $-2^{15} \sim 2^{15}-1$  ——>
    - 无符号：16个数值位。取值范围  $0 \sim 2^{16}-1$  ——> 0 ~ 65535
- 因此，知道数据类型占用内存的大小，就能算出该类型 无符号数、有符号数 对应的取值范围。

## 数据溢出

## 不常用关键字